Risk-based testing

Unit testing

1. What is your experience with exploratory testing?

During my career as a Software Tester, I have gained extensive hands-on experience with exploratory testing methodologies. In my previous role as a Quality Assurance Engineer at XYZ Company, I used exploratory testing to identify critical defects in the application which were previously missed by our automated testing suites.

1. One instance was when I was working on a feature test for the e-commerce website, where the customer was supposed to check out of a shopping cart after adding products. During exploratory testing, I found that the discount code field did not accept the expected codes. This was a significant issue that the automated tests had failed to catch, and the company had lost considerable revenue due to this.
2. Another instance was when I was testing a feature related to the creation of user profiles. I found that user profiles were getting created with invalid characters, which leads to an error in the system, which could potentially lead to security vulnerabilities if not addressed. The automated tests had not caught this error because of the nature of this particular scenario.

Through these instances and numerous other examples, I have learned that exploratory testing has become an essential aspect of any software testing process. It helps us identify defects that would otherwise go unnoticed by automated testing tools, and discovering those can be critical to the success of the product.

2. How do you prioritize which tests to explore?

As an exploratory tester, prioritizing tests is a crucial part of ensuring that the most important aspects of the product are thoroughly tested. The following are the criteria I use to determine which tests to explore first:

1. Business impact: I prioritize tests based on the potential impact on the business. For example, if a product feature is critical to the success of the business, it will be tested first.

2. Criticality: Another important factor is the criticality of the feature or functionality. For instance, if a certain application feature is essential and could lead to significant financial loss, I prioritize testing it early on.
3. Risk level: High-risk areas are prioritized first to ensure that any risks are minimized. If a particular feature has failed in the past or is known to be prone to issues, it will be tested early on.
4. Customer impact: I prioritize tests based on the customer experience. For example, if a feature directly impacts the customer experience, it will be tested early.
5. Frequency of use: The most frequently used features of the application will be prioritized so that any issues that may affect the user's experience can be resolved quickly.
6. Defect history: I go through previous defect reports to identify any recurring issues and prioritize testing these areas to prevent any future issues.
7. Time constraints: I work closely with the project manager and development team to determine any upcoming deadlines and prioritize tests based on which features need to be released first.
8. Test coverage: I prioritize tests based on which areas of the application have not been tested extensively in previous testing cycles.
9. Resource availability: I evaluate the available resources and create a testing plan that optimizes the available resources while ensuring that the priority tests are covered.
10. Effective communication: Lastly, I prioritize tests based on the importance of effectively communicating results to the development team. Feedback loop between the two parties is critical for timely delivery and robust products.

Using these criteria, I can prioritize which tests to explore first, ensuring that the product is thoroughly tested while optimizing available resources and delivering meaningful results to the development team.

3. Can you walk me through your process for exploratory testing?

As a QA Engineer, my process for exploratory testing involves the following steps:

1. **Identifying test scenarios**: I start by reviewing the user stories and requirements for the feature being tested. I then brainstorm potential test scenarios based on these requirements and any knowledge I may have about the product or the user.
2. **Planning the approach**: Once I have identified the test scenarios, I prioritize them and develop a testing plan. I determine the scope, the testing environment and the type of tests that should be conducted.
3. **Executing tests**: I then execute the testing plan, performing the tests according to the priority order. I document any issues that arise, including the steps to reproduce and the severity of the issue.
4. **Analyze results:** After completing the testing, I analyze the results to determine if the feature met the requirements and if there are any issues that need to be resolved. I validate that the results are accurate and that they are meeting up to the expectations.
5. **Feedback**: Finally, I provide feedback to the development team, highlighting any issues that need to be addressed and any improvements that can be made to the feature. I also provide suggestions on the process involved in the testing plan since this can also help to improve the overall product quality.

Throughout the process, I use test case management software to track the results of tests I have executed. For example, in my previous role, I successfully identified five major UI bugs in a new feature through exploratory testing. By finding these issues early in the development lifecycle, and providing detailed documentation on how to reproduce the bugs, helped in decreasing the bugs in the product by 50% over the next two sprints. Also, by updating the testing plan, we were able to eliminate most of the design bugs and server issues in the final product.

4. What strategies do you use to identify defects during exploratory testing?

During exploratory testing, I employ a variety of strategies to identify defects. One key strategy is to use mind maps to organize my exploration process. By visualizing the test coverage and mapping out the testing areas, I can ensure that I am considering all possible scenarios and paths through the software.

In addition to mind maps, I also use heuristic testing methods. By applying a set of heuristics to the software, I can quickly identify areas that may pose a higher risk for defects. For example, I may focus on testing areas of

the software that are especially complex or that have undergone recent changes.

I also use tools to assist in my defect identification. For example, I use both automation tools and manual testing approaches to gather data on the software's behavior. By examining the results of automated tests and performing manual testing in areas that require a human touch, I can identify defects faster and more accurately.

I'm proud to say that one notable example of the effectiveness of these strategies was when I identified a critical defect in a mobile app during exploratory testing. The app was intended to work on both iOS and Android devices, but I found that it was crashing on Android devices under specific conditions. Through mind mapping, heuristic testing, and the use of tools, I was able to isolate the root cause of the issue and communicate it to the development team. This resulted in a timely fix that prevented widespread negative user experiences and saved the company an estimated $50,000 in lost revenue.

5. What tools do you prefer to use for exploratory testing?

During my exploratory testing sessions, I prefer to use a combination of tools that best suits the project requirements.

1. **Bug Tracking System:** I have experience working with JIRA, Redmine, and Trello, which help in documenting and tracking issues found during exploratory testing sessions. In my previous project, I was able to identify and report 60 critical bugs that were promptly fixed and tested, resulting in an overall improvement of the quality of the project.
2. **Browser Dev Tools:** I use Chrome Developer Tools, Firebug, and Safari Web Inspector to examine HTML, CSS, and JavaScript, as well as user session information, network traffic, and page loading speed. In a recent project, I found that optimizing the page load speed using these tools resulted in a dramatic decrease in bounce rates and an increase in page retention time for our users.
3. **Test Management System:** I like to use TestRail, which helps me manage test cases, create test plans and organize test runs. This tool also provides clear test results and statistics, which helps me track my progress and document my findings. In a previous project, I used TestRail to document my exploratory testing sessions, which gave the

team a clear picture of the project's quality and helped us improve our testing strategy.

Overall, I am comfortable using various tools to perform exploratory testing and can adapt to new ones based on the project requirements. I am always on the lookout for new and better tools to help me improve and streamline my testing processes.

6. How do you handle time constraints while exploring a feature?

When dealing with time constraints while exploring a feature, I follow a structured approach to maximize efficiency:

1. Prioritize scenarios: I analyze the feature and identify scenarios that are critical to the user experience or business requirements. I then focus on testing those scenarios first to ensure they are functioning as expected.
2. Divide testing: If the feature has multiple components or modules, I break down the testing into manageable chunks. This helps me allocate my time more efficiently and ensures I cover all key areas.
3. Utilize automation: I leverage automated testing tools wherever possible to save time on repetitive tasks. For instance, I create scripts to run regression testing, freeing up my time to focus on more exploratory testing.
4. Collaborate with stakeholders: I actively involve stakeholders such as developers, product owners, or project managers in the testing process, seeking their input on potential issues or risks. This helps me prioritize testing efforts and improve the overall quality of the feature.

These methods have proven to be effective within my previous roles. For instance, while testing a new feature for a mobile application under a tight deadline, I followed these steps to cover high priority scenarios and identified crucial defects which were addressed and resolved quickly. As a result, the feature was delivered on time and with high quality.

7. How do you ensure that testing is thorough while exploring?

During exploratory testing, I make sure to follow a well-defined testing plan. I start by understanding the goals of the testing phase and defining the testing scope.

1. First, I create a checklist of important test scenarios and test cases that need to be executed. For example, in one project, I had to verify that the system under test was working as expected for a large dataset. I created a set of edge test cases to confirm that the system could handle the maximum amount of data with ease.
2. Next, I communicate with other team members to understand their testing priorities and areas of focus. This way, we could have a complementary approach that ensures nothing was left out.
3. I also use various testing tools and techniques to ensure that I'm testing as thoroughly as possible. For example, I'll use keyword-driven testing to automate repetitive testing steps, leaving me more time to explore complex scenarios.
4. Moreover, I continuously evaluate the quality of the test cases and update them as necessary. This keeps testing fresh and helps prevent overlooking any potential flaws.

As a result of these methods, I can confidently say that I have been able to identify over 90% of critical defects during testing. In one project, I was able to find and log over 50 defects, which saved countless hours and saved my team from delivering products that did not meet the highest standards of quality.

8. Can you provide an example of a difficult problem you found while exploring a product?

During one of my exploratory testing sessions, I came across a critical problem with a website's checkout process. After selecting multiple items for purchase, the "Complete Order" button was not functioning properly, resulting in customers being unable to complete their orders.

1. To reproduce the bug, I attempted to make a purchase multiple times using different payment methods and browsers.
2. Next, I accessed the website's codebase and performed a thorough analysis of the checkout process, including reviewing the code and running various tests.
3. Upon reviewing the code, I discovered that there was a coding error that resulted in the "Complete Order" button redirecting customers to an incorrect page.
4. Using my skills in troubleshooting, I quickly determined the cause of the problem and suggested a fix to the development team.

5. With the development team's approval, I implemented a code fix and tested the new changes to ensure the issue was permanently resolved.
6. After implementation, I monitored the website's analytics to confirm that the issue had been resolved and the checkout process was operating correctly.
7. As a result of my quick identification and solution of the problem, the website's sales increased by 15% over the following month.

This experience taught me the importance of exploratory testing in uncovering critical issues, and how effective communication and collaboration is crucial between development and testing teams to solve issues quickly and efficiently.

9. What is your approach for collaborating with developers during exploratory testing?

Collaborating with developers during exploratory testing is essential because it provides an opportunity to catch defects early in the development cycle. At the start of each project, I coordinate with the development team to establish a shared understanding of the product, its intended behavior, and objectives.

1. **Initial Discussion:** I hold an initial discussion with the developers to get an overview of the functionality and architecture of the software. I gather all the necessary information that I can use for the exploratory testing process, including the user flow, dependencies, backend structure, and so on.
2. **Defect Reporting:** While performing exploratory testing, I use defect tracking tools or agile management software to report any issues that I come across. A common practice is to assign each reported defect to the responsible developer or development team. By doing so, they can follow-up and investigate the issue right away, allowing for rapid resolution.
3. **Collaborative Retesting:** When a defect is fixed, I retest it and verify that the issue has been resolved. This process ensures that the result aligns with the intended design and follows the necessary requirements.
4. **Continuous Feedback:** I provide continuous feedback to developers about the quality of the product during exploratory testing. Providing feedback early in the software development life cycle helps to detect

defects early and correct them promptly, saving the company time and money.

5. **Agile Approach:** I follow an agile approach that places great importance on collaboration between development and testing teams. This approach enables the development team to write clean, high-quality code while the testing team can catch potential issues before they escalate into more serious problems.

Overall, my approach to collaborating with developers during exploratory testing allows for a faster and more thorough testing process. By working in partnership with the development team, we can reduce defects and achieve a high level of product quality.

10. Can you explain how you incorporate exploratory testing into your test plan?

Exploratory testing is an essential part of my testing plan, and it involves much more than just clicking around haphazardly. When performing exploratory testing, I focus on looking for anomalies, edge cases, and behaviors that may not be immediately apparent. I also use heuristics and models to guide my testing.

1. First, I familiarize myself with the product and its intended use, making sure I understand its specifications and goals.
2. Next, I start executing tests with a focus on areas that are most critical to the product's functionality or most likely to present issues. I look for unexpected or inconsistent behavior.
3. I also use experiments to test the application in various configurations and hardware to simulate usage in real life.
4. If I find an issue, I test the surrounding functionalities to ensure that the problem is not due to an upstream process.
5. Once I have found an issue, I report it to the project lead and make sure that it gets resolved. If necessary, I provide additional background or data to support my findings.
6. I also ensure that I document my test results systematically and continuously update my test plan to incorporate any lessons learned.
7. Finally, I test the product again to ensure that the bugs fixed during testing do not introduce additional issues.

Through this process, I have found several critical bugs that could have affected users severely. For example, I uncovered a network connectivity

issue that prevented users in specific geographic locations from accessing critical features. By addressing this issue, we were able to increase our user base significantly and gain valuable insight into our product's usability.

Conclusion

These exploratory testing interview questions and answers will prepare any remote QA Engineer for a successful interview in 2023. However, getting the job requires more than just interview skills. It's crucial to have a well-written cover letter that highlights your skills and experience. Check out our guide on writing a compelling cover letter for quality assurance engineers to help you stand out. Additionally, having an impressive resume is crucial when applying for a job. Our guide on writing a resume for QA Engineers will help you craft a strong and effective resume. Finally, don't forget to check out Remote Rocketship's job board for remote QA Engineer positions. We wish you the best of luck in your job search!

Find your next remote QA Engineer job by visiting our job board: Remote QA Engineer Jobs

Looking for a remote job? Search our job board for 100,000+ remote jobs

Search Remote Jobs

Receive Emails For Remote Jobs

Discover 100,000+ Remote Jobs!

Join now to unlock all jobs

Discover hidden jobs

We scan the internet everyday and find jobs not posted on LinkedIn or